KU LEUVEN

Labo 3 - CUDA

Sander De Stercke (MEIECFS)
Isaak Vervenne (MEISWE)

Februari 2026

1 Wat is er gerealiseerd?

Dit labo onderzoekt de impact van warp-partitionering op de executietijd van een image-inverter. Daarnaast zal er ook geëxperimenteerd worden met warp divergence en hoe deze de executietijd beïnvloedt.

In het eerste deel hebben we een kernel geschreven die een afbeelding inverteert. De kleurcomponenten worden aangepast met de bewerking: $x = 255 - x$. We hebben ook grid-striding toegepast, waardoor de GPU grote hoeveelheden data kan verwerken, zelfs als er minder threads worden toegewezen dan data-elementen.

In het tweede deel wordt thread divergence geanalyseerd. We schreven een kernel waarbij op de rode kleurcomponenten een complexe bewerking (1D Gaussiaanse blur) wordt uitgevoerd, terwijl de groene en blauwe componenten eenvoudigweg worden geïnverteerd. Standaard is RGB-data van een afbeelding interleaved (RGBRGBRGB...), wat ervoor zorgt dat de threads binnen één warp verschillende instructies moeten uitvoeren, met thread divergence als gevolg. Vervolgens hebben we een planaire structuur (RR...R GGG...G BBB...B) geïmplementeerd en geëvalueerd om deze divergence te vermijden en de impact hiervan op de executietijd te meten.

2 Problemen

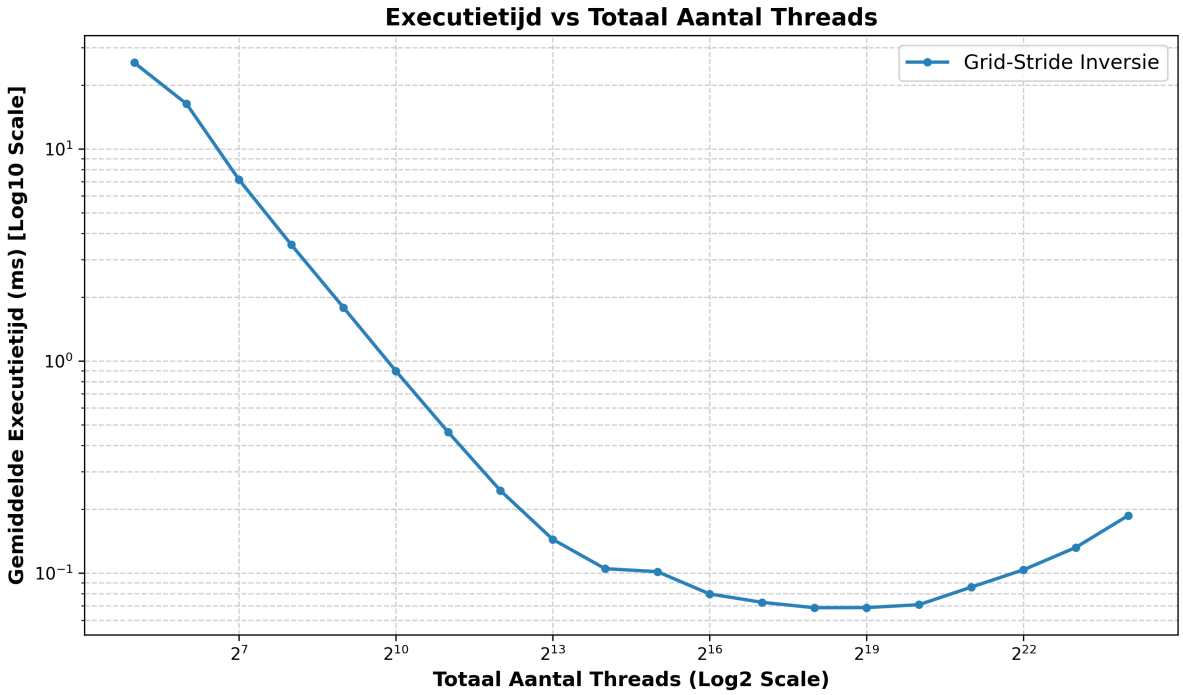
Tijdens de implementatie moesten we rekening houden met enkele valkuilen en randgevallen:

- **Randvoorwaarden bij Gaussian blur:** Bij het berekenen van waarden aan de randen van de afbeelding ontstonden potentiële out-of-bounds memory accesses. Dit hebben we opgelost door *clamping* (via ternaire operatoren) toe te passen op de indexen, zodat we veilig de dichtstbijzijnde randwaarde overnemen.
- **Opstartoverhead GPU:** Bij het timen van de kernel zagen we aanvankelijk onstabiele meetwaarden, omdat de allereerste aanroep van een kernel aanzienlijk trager is vanwege de context initialization van de GPU. Door de kernels eerst een ongetimede "warm-up"te geven, hebben we betrouwbare gemiddelde executietijden bekomen.
- **Geheugentransfers in de meting:** In eerste instantie namen we per ongeluk de `cudaMemcpy` (het kopiëren van data van Host naar Device via de PCIe-bus) mee in onze tijdmetingen. Hierdoor werd de pure executietijd van de kernels gemaskeerd door de enorme I/O-overhead. Dit hebben we gecorrigeerd door de `cudaEventRecord`-timers uitsluitend rond de kernel launch te plaatsen.

3 Vragen en Analyse

3.1 Hoe beïnvloedt het aantal threads de executietijd?

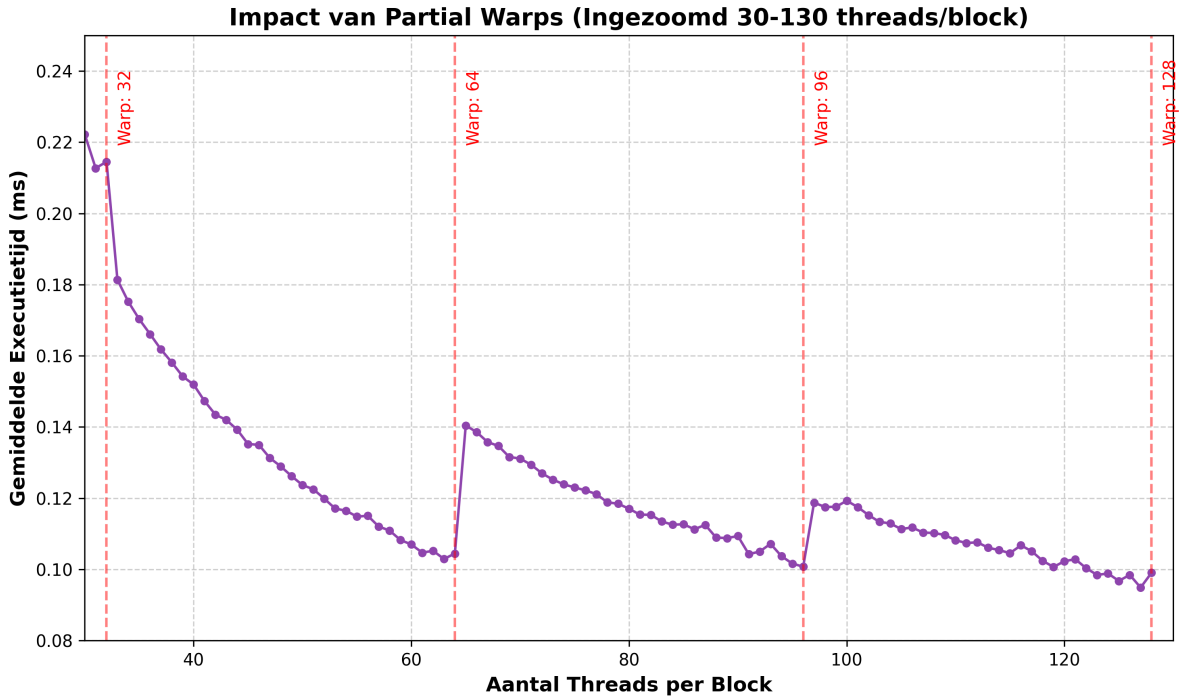
Threads worden beheerd in warps, vaste groepen van 32 threads. Via grid-striding is het mogelijk om met een klein aantal threads alsnog de hele afbeelding te verwerken, doordat elke thread iteratief meerdere elementen voor zijn rekening neemt.



Figuur 1: Executietijd versus het totaal aantal threads bij het inverteren van de afbeelding (Log-Log scale).

In Figuur 1 is de impact van het totaal aantal opgestarte threads goed zichtbaar. Naarmate de parallellisatiegraad toeneemt, zien we een drastische daling in de executietijd. Rond de 2^{18} à 2^{20} threads (ongeveer 250.000 tot 1 miljoen threads) bereiken we een optimaal punt waarbij de GPU volledig verzadigd is. Gaan we hier nog ver boven zitten, dan neemt de overhead van het schedulen en creëren van onnodig veel blocken de overhand en stijgt de tijd weer lichtjes.

Naast het totale aantal threads, is ook de verdeling van de threads in *blocks* van groot belang. Het aantal toegewezen warps per block wordt berekend door het aantal threads per block te delen door 32. Om het effect hiervan te bestuderen, hebben we de blockgrootte laten variëren van 1 tot 128 threads per block (zie Figuur 2).



Figuur 2: Impact van Partial Warps bij variërende thread block groottes (ingezoomd 30-130 threads).

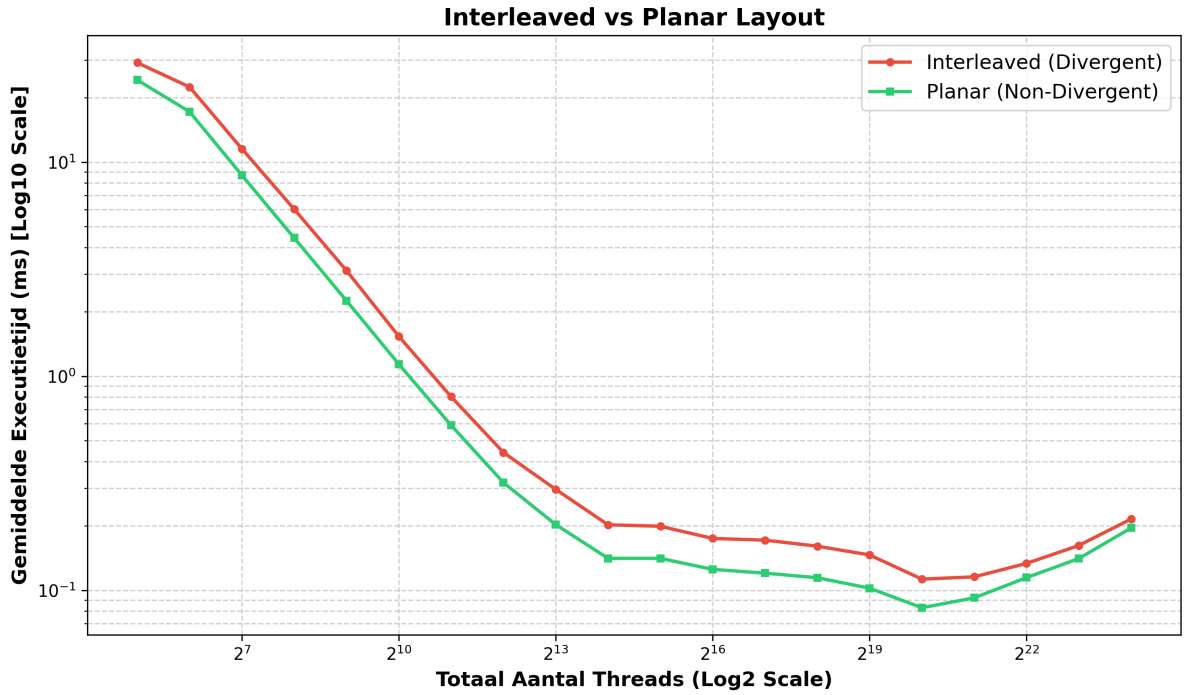
In deze detailweergave is een duidelijk zaagtandpatroon zichtbaar net na veelvouden van 32. Wanneer de blockgrootte nét boven een veelvoud van 32 komt (bijvoorbeeld bij 33 threads), forceert de GPU de allocatie van een extra, onvolledige warp. Een aanroep met bijvoorbeeld 33 threads per block vereist daardoor twee warps, oftewel 64 actieve cores. Hierbij blijven er echter 31 threads binnen de tweede warp onbenut, wat resulteert in een plotselinge, relatieve piek in de executietijd door hardware-onderbenutting. Naarmate deze “extra” warp verder wordt opgevuld met functionele threads, daalt de executietijd weer. Dit bewijst dat voor een optimale hardware-benutting het aantal threads per block altijd een exact veelvoud van 32 moet zijn.

3.2 Wat is de impact van thread divergence op de executietijd?

De threads binnen één warp werken volgens het SIMT-principe (Single Instruction, Multiple Threads) en voeren dus gelijktijdig dezelfde instructie uit. Als threads binnen een warp een verschillend executiepad kiezen (divergeren), bijvoorbeeld door een verschil in rekencomplexiteit, moet de warp de ongebruikte threads tijdelijk op ’idle’ zetten en wachten tot het complexe pad is afgewerkt.

We veronderstelden dat we thread divergence konden elimineren door de afbeeldingsdata om te vormen naar een planaire layout (eerst alle R-waarden, dan alle G, dan alle B). Hierdoor zal een volledige warp óf enkel de zware Gaussiaanse blur uitvoeren (R), óf enkel

de snelle inversie (G of B). Binnen de warp is er dan geen vertakking meer.



Figuur 3: Impact van Thread Divergence (Interleaved) vs Memory Locality (Planar Layout).

Onze metingen in Figuur 3 bevestigen deze theorie overduidelijk. De groene lijn (Planar, Non-Divergent) ligt voor nagenoeg alle thread-configuraties lager dan de rode lijn (Interleaved, Divergent). Dit toont aan dat warps op instructieniveau efficiënter worden afgehandeld in de planaire implementatie, omdat ze niet gehinderd worden door idle clock cycles. Het sorteren van de data op de CPU kost logischerwijs tijd, maar voor de pure GPU-kernel executie levert het uitsluiten van divergence een onmiskenbare prestatiewinst op.

4 Bijlage: C++ Code

```cpp
#include <cuda_runtime.h>
#include <device_launch_parameters.h>
#include <iostream>
#include <vector>
#include <fstream>
#include <string>
#include <iomanip>

const int NUM_ITERATIONS = 5;

// ============================================================
// PPM IMAGE I/O FUNCTIONS
// ============================================================
bool readPPM(const char *filename, std::vector<unsigned char> &image, int &width, int &height) {
    std::ifstream file(filename, std::ios::binary);
    if (!file) {
        std::cerr << "Error: Could not open " << filename << "\n";
        return false;
    }
    std::string magic;
    file >> magic;
    if (magic != "P6") return false;

    char c;
    file >> c;
    while (c == '#') {
        file.ignore(256, '\n');
        file >> c;
    }
    file.unget();

    int max_val;
    file >> width >> height >> max_val;
    file.ignore(256, '\n');

    image.resize(width * height * 3);
    file.read(reinterpret_cast<char*>(image.data()), image.size());
    return true;
}

bool writePPM(const char *filename, const std::vector<unsigned char> &image, int width, int height) {
    std::ofstream file(filename, std::ios::binary);
    if (!file) return false;
    file << "P6\n" << width << " " << height << "\n255\n";
    file.write(reinterpret_cast<const char*>(image.data()), image.size());
    return true;
}

// ============================================================
// GPU KERNELS
// ============================================================

// Deel 1: Simpele Inversie
__global__ void invert_image_strided(unsigned char *img, int num_elements) {
    int index = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = blockDim.x * gridDim.x;

```

```
58      for (int i = index; i < num_elements; i += stride) {
59          img[i] = 255 - img[i];
60      }
61  }
62
63  // Deel 2: Interleaved (Divergent)
64  // 1 Thread verwerkt 1 kleurcomponent (R, G of B). Hierdoor ontstaat divergence.
65  __global__ void process_divergent_rgb(unsigned char *img, int num_elements) {
66      int index = blockIdx.x * blockDim.x + threadIdx.x;
67      int stride = blockDim.x * gridDim.x;
68
69      for (int i = index; i < num_elements; i += stride) {
70          int channel = i % 3; // 0 = R, 1 = G, 2 = B
71
72          if (channel == 0) {
73              // Complexe bewerking: 1D Gaussian blur voor R
74              int r_0  = img[i];
75              int r_m1 = (i >= 3) ? img[i - 3] : r_0;
76              int r_m2 = (i >= 6) ? img[i - 6] : r_0;
77              int r_p1 = (i + 3 < num_elements) ? img[i + 3] : r_0;
78              int r_p2 = (i + 6 < num_elements) ? img[i + 6] : r_0;
79
80              img[i] = (unsigned char)(0.1f * r_m2 + 0.25f * r_m1 + 0.5f * r_0 + 0.25f
                    * r_p1 + 0.1f * r_p2);
81          } else {
82              // Simpele bewerking: Inversie voor G en B
83              img[i] = 255 - img[i];
84          }
85      }
86  }
87
88  // Deel 2: Planar Layout (Non-Divergent)
89  // Array layout: Eerst alle R, dan alle G, dan alle B.
90  __global__ void process_planar_layout(unsigned char *img_planar, int num_pixels, int
        num_elements) {
91      int index = blockIdx.x * blockDim.x + threadIdx.x;
92      int stride = blockDim.x * gridDim.x;
93
94      for (int i = index; i < num_elements; i += stride) {
95          if (i < num_pixels) {
96              // Complexe bewerking: Omdat R allemaal achter elkaar zit, gaat een hele
                    warp hierin
97              int r_0  = img_planar[i];
98              int r_m1 = (i >= 1) ? img_planar[i - 1] : r_0;
99              int r_m2 = (i >= 2) ? img_planar[i - 2] : r_0;
100             int r_p1 = (i + 1 < num_pixels) ? img_planar[i + 1] : r_0;
101             int r_p2 = (i + 2 < num_pixels) ? img_planar[i + 2] : r_0;
102
103             img_planar[i] = (unsigned char)(0.1f * r_m2 + 0.25f * r_m1 + 0.5f * r_0 +
                      0.25f * r_p1 + 0.1f * r_p2);
104         } else {
105             // Simpele bewerking: Warps voor G en B
106             img_planar[i] = 255 - img_planar[i];
107         }
108     }
109 }
110
111 // ============================================================
112 // HOST CODE (CPU)
113 // ============================================================
114
```

```
115 int main() {
116     int width, height;
117     std::vector<unsigned char> h_img;
118 
119     if (!readPPM("anna.ppm", h_img, width, height)) return -1;
120 
121     int num_pixels = width * height;
122     int num_elements = num_pixels * 3;
123     size_t size_bytes = num_elements * sizeof(unsigned char);
124 
125     std::cout << "Loaded␣Image␣'anna.ppm':␣" << width << "x" << height << "␣(" <<
        num_elements << "␣elements)\n";
126 
127     unsigned char *d_img;
128     cudaMalloc(&d_img, size_bytes);
129 
130     cudaEvent_t start, stop;
131     cudaEventCreate(&start);
132     cudaEventCreate(&stop);
133 
134     // ============================================================
135     // PART 1: Dense Thread Scaling Analysis
136     // ============================================================
137     std::cout << "\n---␣PART␣1:␣Dense␣Thread␣Scaling␣Analysis␣---\n";
138     std::cout << "Total_Threads␣|␣Blocks␣|␣Threads/Block␣|␣Avg␣Time␣(ms)\n";
139 
140     for (int total_threads = 32; total_threads <= 16777216; total_threads *= 2) {
141         int threads_per_block = 256;
142         if (total_threads < 256) {
143             threads_per_block = total_threads;
144         }
145         int blocks = (total_threads + threads_per_block - 1) / threads_per_block;
146 
147         float total_time = 0.0f;
148 
149         // Warmup (zonder timer)
150         invert_image_strided<<<blocks, threads_per_block>>>(d_img, num_elements);
151         cudaDeviceSynchronize();
152 
153         for(int iter = 0; iter < NUM_ITERATIONS; iter++) {
154             // Let op: Memcpy BUITEN de timer!
155             cudaMemcpy(d_img, h_img.data(), size_bytes, cudaMemcpyHostToDevice);
156 
157             cudaEventRecord(start);
158             invert_image_strided<<<blocks, threads_per_block>>>(d_img, num_elements);
159             cudaEventRecord(stop);
160             cudaEventSynchronize(stop);
161 
162             float ms = 0;
163             cudaEventElapsedTime(&ms, start, stop);
164             total_time += ms;
165         }
166         std::cout << total_threads << "␣|␣" << blocks << "␣|␣" << threads_per_block
            << "␣|␣" << (total_time / NUM_ITERATIONS) << "\n";
167     }
168 
169     // ============================================================
170     // PART 1B: Partial Warps (Threads Per Block Scaling)
171     // ============================================================
172     std::cout << "\n---␣PART␣1B:␣Partial␣Warps␣Analysis␣---\n";
173     std::cout << "Threads/Block␣|␣Avg␣Time␣(ms)\n";
```

```
174
175     // We testen van 1 tot 128 threads per block om de 32-thread sprongen (warps)
            goed te zien.
176     for (int tpb = 1; tpb <= 128; tpb++) {
177         // We zorgen dat we altijd de hele afbeelding verwerken
178         int blocks = (num_elements + tpb - 1) / tpb;
179
180         float total_time = 0.0f;
181
182         // Warmup
183         invert_image_strided<<<blocks, tpb>>>(d_img, num_elements);
184         cudaDeviceSynchronize();
185
186         for(int iter = 0; iter < NUM_ITERATIONS; iter++) {
187             cudaMemcpy(d_img, h_img.data(), size_bytes, cudaMemcpyHostToDevice);
188
189             cudaEventRecord(start);
190             invert_image_strided<<<blocks, tpb>>>(d_img, num_elements);
191             cudaEventRecord(stop);
192             cudaEventSynchronize(stop);
193
194             float ms = 0;
195             cudaEventElapsedTime(&ms, start, stop);
196             total_time += ms;
197         }
198         std::cout << tpb << "␣|␣" << (total_time / NUM_ITERATIONS) << "\n";
199     }
200
201     // ============================================================
202     // PART 2: Dense Divergence vs Planar Analysis
203     // ============================================================
204     std::cout << "\n---␣PART␣2:␣Dense␣Divergence␣Analysis␣---\n";
205     std::cout << "Total_Threads␣|␣Interleaved␣(ms)␣|␣Planar␣(ms)\n";
206
207     // 1. Bereid Planar Layout voor op CPU (formaat: RR...RGGG...GBB...B)
208     std::vector<unsigned char> h_planar(num_elements);
209     for(int i = 0; i < num_pixels; i++) {
210         h_planar[i]                     = h_img[i*3];       // Red
211         h_planar[num_pixels + i]        = h_img[i*3 + 1];   // Green
212         h_planar[2 * num_pixels + i]    = h_img[i*3 + 2];   // Blue
213     }
214
215     unsigned char *d_planar;
216     cudaMalloc(&d_planar, size_bytes);
217
218     // Warmup kernels
219     int wu_threads = 256;
220     process_divergent_rgb<<<1, wu_threads>>>(d_img, num_elements);
221     process_planar_layout<<<1, wu_threads>>>(d_planar, num_pixels, num_elements);
222     cudaDeviceSynchronize();
223
224     for (int total_threads = 32; total_threads <= 16777216; total_threads *= 2) {
225         int threads_per_block = 256;
226         if (total_threads < 256) threads_per_block = total_threads;
227         int blocks = (total_threads + threads_per_block - 1) / threads_per_block;
228
229         // Meting 1: Interleaved (Divergent)
230         float total_interleaved = 0.0f;
231         for(int iter = 0; iter < NUM_ITERATIONS; iter++) {
232             cudaMemcpy(d_img, h_img.data(), size_bytes, cudaMemcpyHostToDevice);
233
```

```
234             cudaEventRecord(start);
235             process_divergent_rgb<<<blocks, threads_per_block>>>(d_img, num_elements)
                    ;
236             cudaEventRecord(stop);
237             cudaEventSynchronize(stop);
238 
239             float ms = 0;
240             cudaEventElapsedTime(&ms, start, stop);
241             total_interleaved += ms;
242         }
243 
244         // Meting 2: Planar (Non-Divergent)
245         float total_planar = 0.0f;
246         for(int iter = 0; iter < NUM_ITERATIONS; iter++) {
247             cudaMemcpy(d_planar, h_planar.data(), size_bytes, cudaMemcpyHostToDevice)
                    ;
248 
249             cudaEventRecord(start);
250             process_planar_layout<<<blocks, threads_per_block>>>(d_planar, num_pixels
                    , num_elements);
251             cudaEventRecord(stop);
252             cudaEventSynchronize(stop);
253 
254             float ms = 0;
255             cudaEventElapsedTime(&ms, start, stop);
256             total_planar += ms;
257         }
258 
259         std::cout << total_threads << "␣|␣" << (total_interleaved/NUM_ITERATIONS) <<
                "␣|␣" << (total_planar/NUM_ITERATIONS) << "\n";
260     }
261 
262     // Cleanup
263     cudaFree(d_img);
264     cudaFree(d_planar);
265     cudaEventDestroy(start);
266     cudaEventDestroy(stop);
267 
268     return 0;
269 }
```

Listing 1: L3a.cu CUDA implementatie met Dense Scaling en Divergence optimalisaties